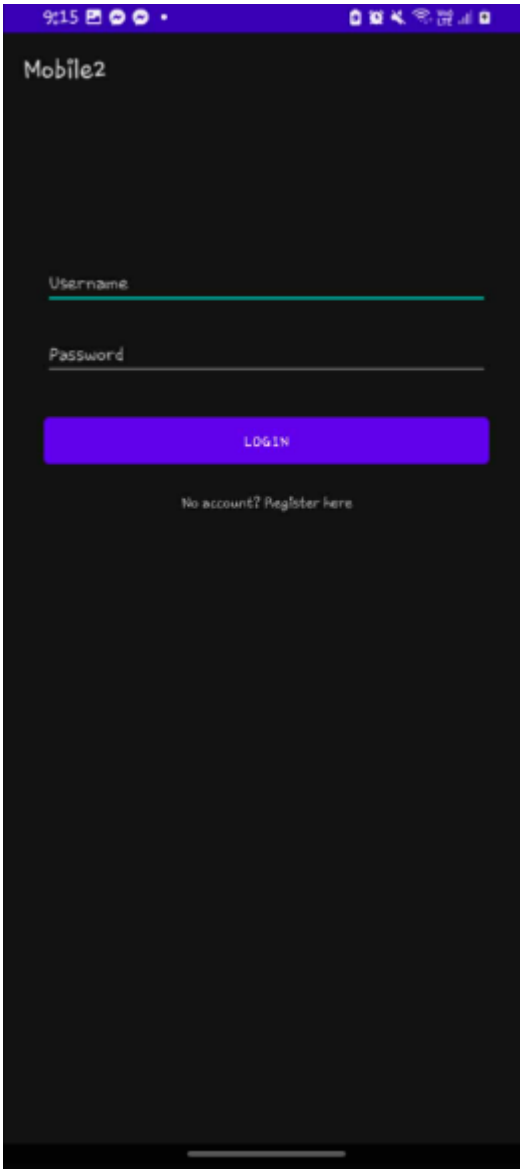
Name: **Chavez, Bliss B**.
Course and Section: IT342 - G2
Project Name: **EbikeRental**
Date: March 30, 2026

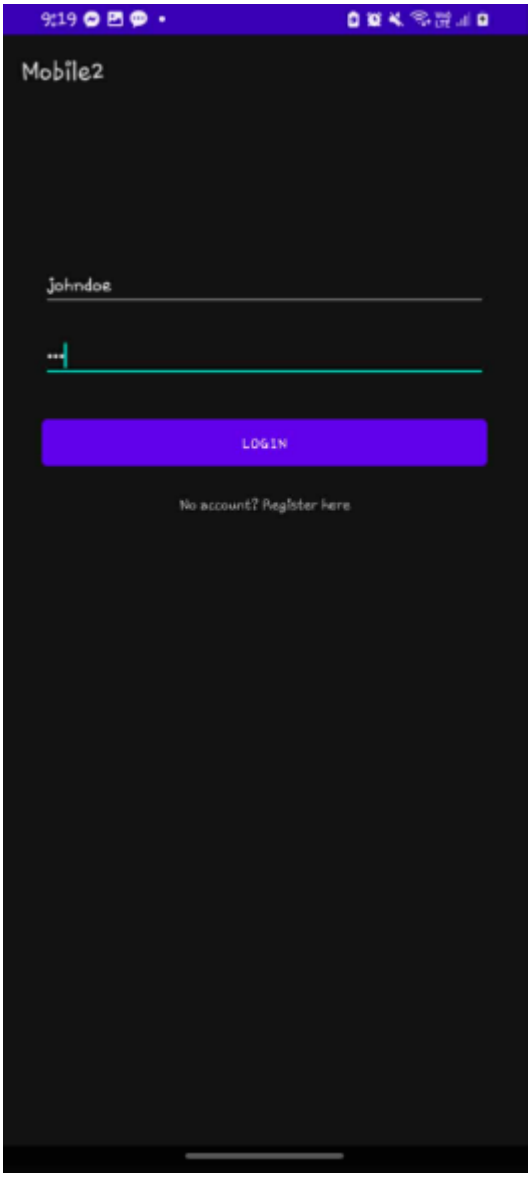
Final Commit for Phase 2

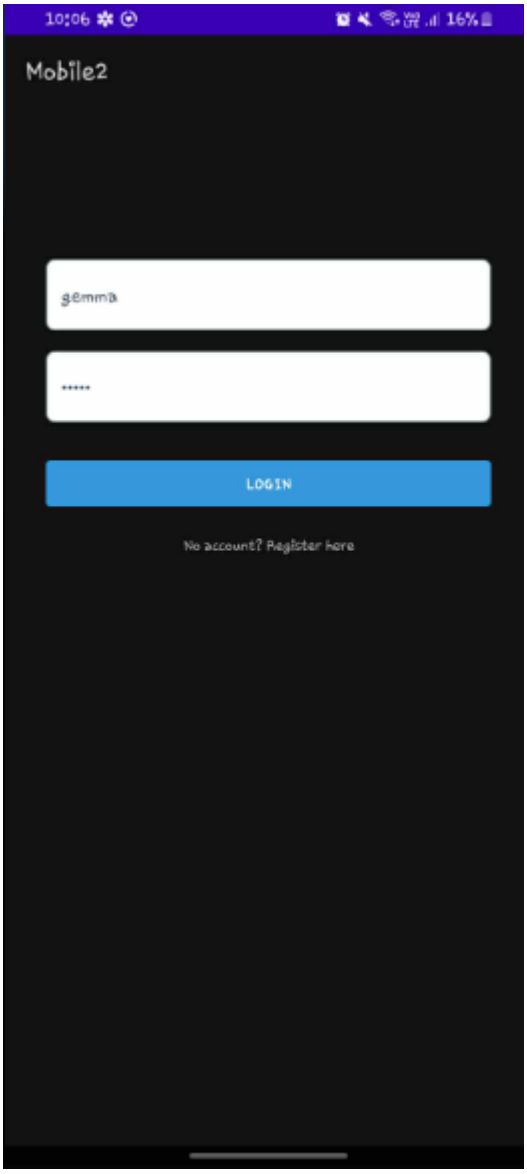
Commit Message:	**IT342 Phase 2 - Mobile Development Completed**
Commit Hash	**a7c0d484b85f6b461293fa2cf27d18e1a7dfe114**
Commit Link	**https://github.com/Bleassy/IT342-Chavez-EbikeRental/commit/a7c0d484b85f6b461293fa2cf27d18e1a7dfe114**

SCREENSHOT

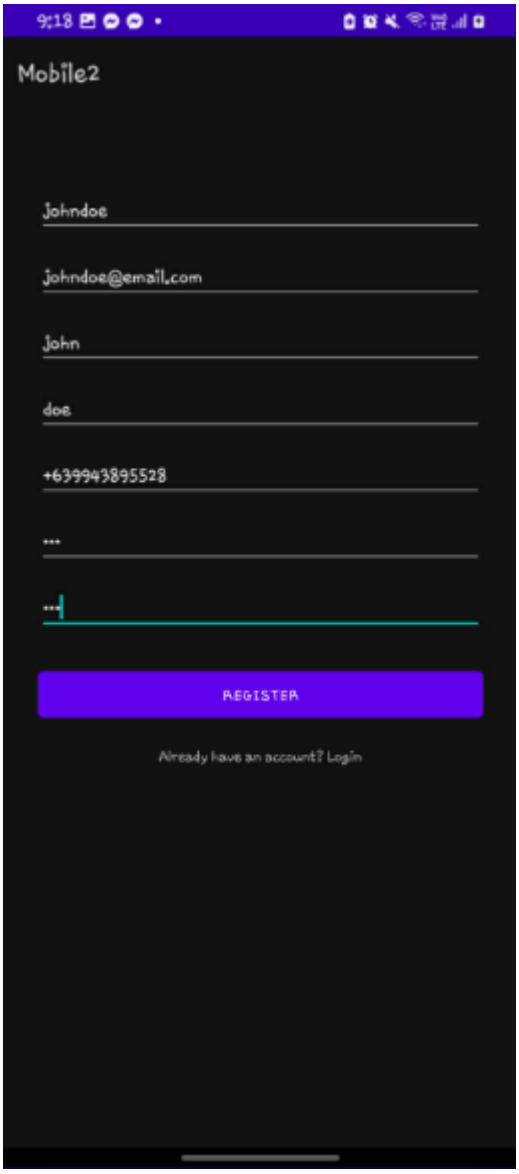
LOGIN

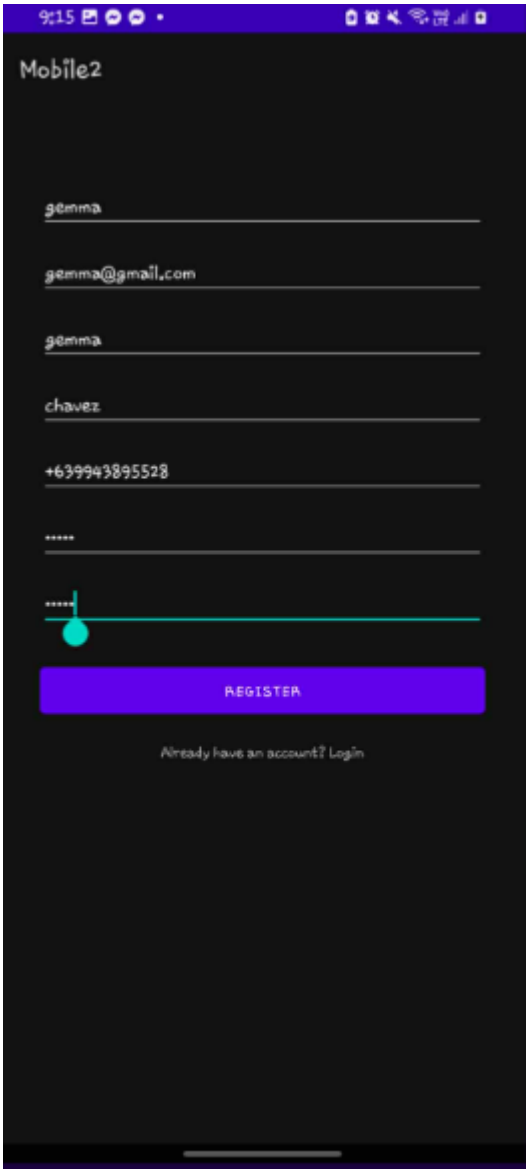

9:15
Mobile2
Username
Password
LOGIN
No account? Register here


9:19
Mobile2
Johndoe
LOGIN
No account? Register here

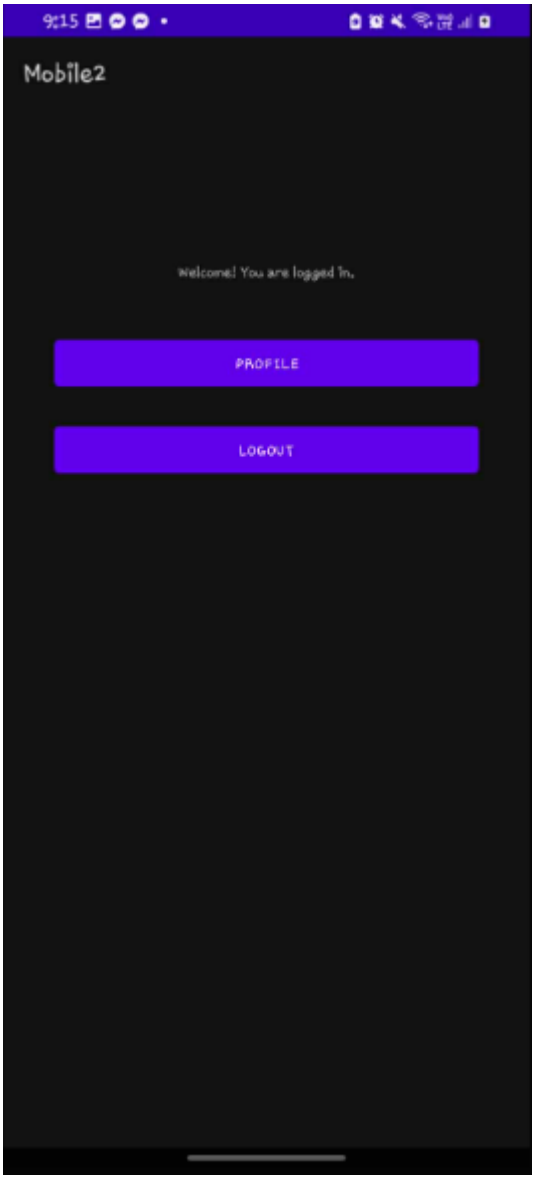


REGISTER

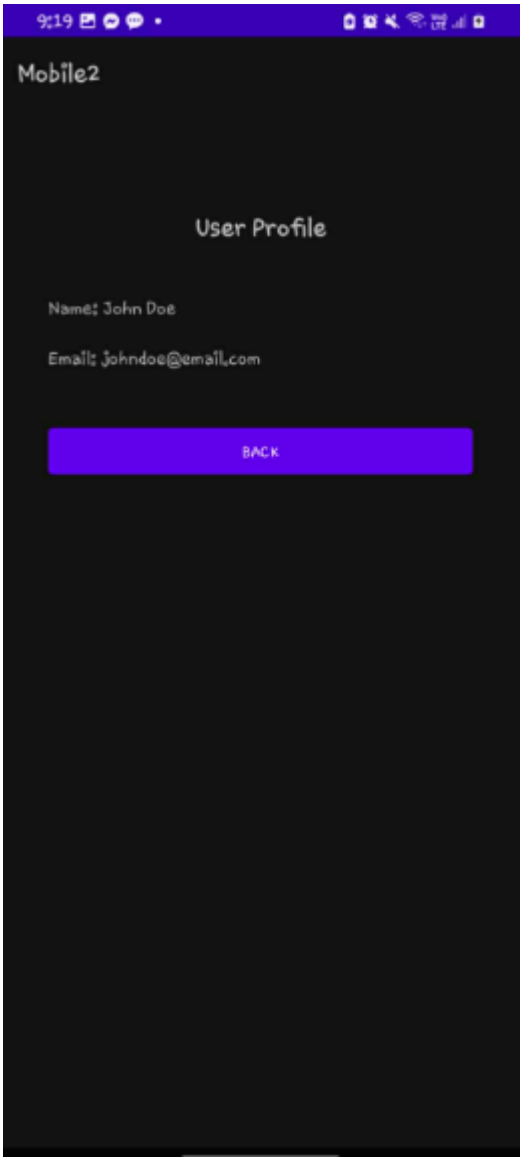

9:18
Mobile2
Johndoe
johndoe@email.com
John
doe
+639943895528
REGISTER
Already have an account? Login



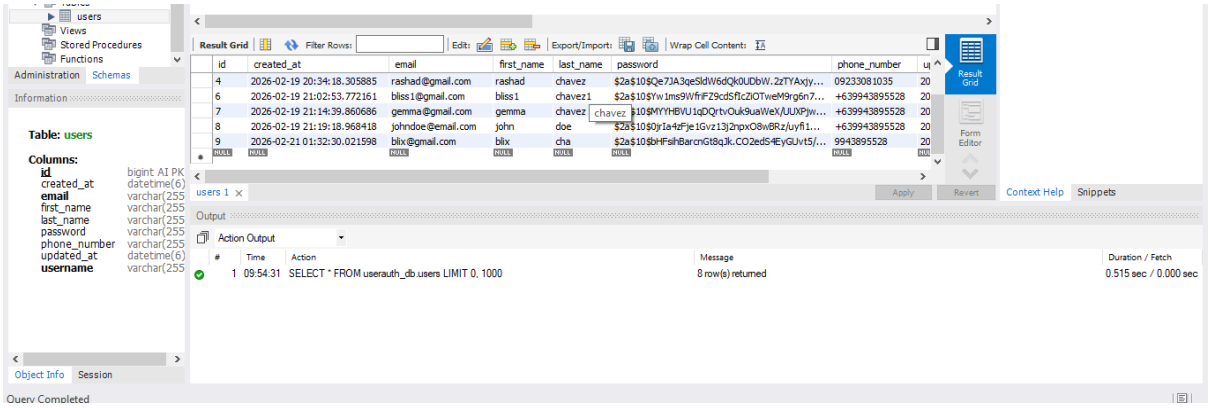
SUCCESSFUL LOGIN



INITIAL PROFILE



DATABASE



SHORT SUMMARY

1. How Registration Works

The registration screen collects the user's First Name, Last Name, Email, and Password. Before sending any request, the app performs client-side validation to ensure no field is left

blank and that the password meets the minimum length requirement. Once validated, the data is packaged into a JSON request body and sent to the backend via a POST request. The backend enforces server-side validation, checking that all required fields are present and that the email is not already taken. If the email is already registered, the backend returns HTTP 409 Conflict and the app displays an error message. On success, the backend returns HTTP 201 Created and the user is redirected to the Login screen.

2. How Login Works

The login screen accepts the user's registered Email and Password. The app first checks that neither field is empty before making a network call. The credentials are sent as a JSON body to the backend login endpoint. The backend looks up the user by email and uses BCryptPasswordEncoder to verify the submitted password against the stored hash. If the credentials are invalid or the email does not exist, the backend returns HTTP 401 Unauthorized and the app displays an "Invalid email or password" error message. On success, the backend returns HTTP 200 OK along with a signed JWT token and the user's profile information. The token is saved locally using SharedPreferences and the user is redirected to the Home screen, which displays their name and email.

3. API Integration Used

The mobile app connects to the existing Phase 1 Spring Boot backend using Retrofit 2 with a Gson Converter for automatic JSON serialization and deserialization. An OkHttp Logging Interceptor is included for debugging request and response bodies during development. The base URL is set to http://10.0.2.2:8080/ which maps to localhost when running on an Android emulator. All network calls are executed using Kotlin Coroutines inside lifecycleScope.launch to keep the UI thread unblocked. Two endpoints are used:

POST /api/auth/register — registers a new user and returns 201 Created with a JWT token

POST /api/auth/login — authenticates the user and returns 200 OK with a JWT token